A MINI PROJECT REPORT

SHREYARTH UNIVERSITY

CARE-GRID

(Doctor Website Maker)

Submitted in partial fulfilment of the requirements for the degree of

Integrated B.Sc. - M.Sc. (CA & IT)

Submitted by

PARTH KADIYA

Enrollment No.: 224005013

School of Computer Science and Application

Shreyarth University, Ahmedabad

Academic Year 2025-26 | May 2026

CERTIFICATE

This is to certify that the project report entitled “Care-Grid (Doctor Website Maker)” has been carried out by Parth Kadiya (Enrollment No. 224005013) under the prescribed academic framework in partial fulfilment of the requirements for the Integrated Master of Science in CA & IT at the School of Computer Science and Application, Shreyarth University, Ahmedabad, during the academic year 2025-26.

The work presented in this report is a bonafide record of the design, implementation, testing and production-readiness activities performed for the Care-Grid platform.

Internal Guide	Head of Department
Name: Prof. Krutik Sijitra Signature: _____	Name: Dr. Nikita Shah Signature: _____

School of Computer Science and Application
Shreyarth University, Ahmedabad

DECLARATION

I hereby declare that the project report entitled “Care-Grid (Doctor Website Maker)”, submitted in partial fulfilment of the Integrated Master of Science in CA & IT to Shreyarth University, Ahmedabad, is an original record of work carried out by me. The report has not been submitted to any other university or institution for the award of any degree or diploma.

All technologies, documentation and external resources used during the project have been appropriately acknowledged. No confidential keys, passwords or deployment secrets are included in this report.

Student Name	Enrollment No.	Signature
Parth Kadiya	224005013	_____

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty members of the School of Computer Science and Application, Shreyarth University, for providing the academic foundation, guidance and encouragement required to complete this project. Their feedback helped shape the project from an initial doctor website generator into a secure, database-driven healthcare portal.

I am thankful to my internal guide and the Head of Department for their support throughout the analysis, development, testing and documentation phases. I also acknowledge my friends and family for their patience and motivation during the project.

Finally, I acknowledge the open-source communities and official documentation of React, Node.js, Express, MongoDB, Cloudinary, Nodemailer, OpenAI, Vercel and Render, which supported the implementation and production-readiness of Care-Grid.

Parth Kadiya | Enrollment No. 224005013

ABSTRACT

Care-Grid is a full-stack MERN platform that enables a doctor to create a responsive professional website by completing a structured onboarding form. The platform generates public-facing website content, manages profile and clinic gallery images, exposes appointment booking to patients, and provides dedicated operational dashboards for doctors and a centralized super administrator.

The system uses React and Vite for the frontend, Node.js and Express for the API layer, MongoDB for persistent storage, Cloudinary for media management, Gmail SMTP for one-time-password delivery, and OpenAI services for grounded administrative assistance and website content. Appointments submitted from a doctor's public website are stored in MongoDB and become visible in both the doctor dashboard and the super-admin dashboard.

Security is treated as a core requirement. Registration, doctor login and super-admin login use purpose-bound email OTP verification. Passwords and OTP values are stored as bcrypt hashes, access tokens are role-bound, password reset invalidates older doctor sessions, API routes are protected with authorization middleware, and validation and rate limiting are enforced across sensitive workflows.

Project	**Outcome**
Care-Grid provides a practical, production-oriented foundation for creating and managing doctor websites, appointments and healthcare portal operations from a single platform.	

TABLE OF CONTENTS

LIST OF FIGURES AND TABLES

CHAPTER 1

Introduction and Project Overview

CARE-GRID: DOCTOR WEBSITE MAKER

1.1 Introduction

Doctors and small clinics increasingly require a trustworthy digital presence, but building and maintaining an individual website, appointment workflow and administrative dashboard generally requires technical expertise and multiple disconnected services. Care-Grid addresses this challenge by offering a single platform that converts structured doctor and clinic information into a modern public website and an operational management environment.

A doctor enters professional details, clinic information, contact information, working schedule, website theme, profile image and gallery images. After verifying the submitted email address using an OTP, Care-Grid stores the information and publishes a unique doctor website. Patients can then view treatments, clinic details, gallery images and directions, and can submit appointment requests.

1.2 Problem Statement

- Independent doctors may not have the technical resources to create and maintain a responsive website.
- Patient appointment requests are often collected through unstructured calls or messages and are difficult to track.
- A doctor requires a private dashboard while a platform owner requires a centralized view across all doctor portals.
- Authentication and account recovery must be secure without making the workflow difficult for non-technical users.
- Public websites, media, administrative data and AI-powered assistance need to work together without exposing sensitive credentials or ungrounded information.

1.3 Project Objectives

1. Enable doctors to generate a professional website through a guided form.
2. Require verified email ownership before creating a doctor portal.
3. Store doctor, appointment, OTP and chatbot records in MongoDB.
4. Provide role-specific doctor and super-admin dashboards with analytics.
5. Support appointment booking and status management.
6. Provide a data-grounded chatbot that responds from admin-panel data.
7. Apply responsive design, validation, secure authentication and production deployment practices.

1.4 Scope

The current scope covers doctor website generation, responsive themes, profile and gallery media, public appointment booking, doctor dashboard operations, super-admin monitoring, grounded chatbot support, OTP-based authentication, password recovery, deployment configuration and health monitoring. The platform does not provide medical diagnosis, online prescriptions, payment processing or a full hospital information management system.

1.5 Key Stakeholders

Stakeholder	Primary needs
Doctor	Create and update website; view and manage appointments; access grounded dashboard assistance

Stakeholder	Primary needs
Patient	View trusted clinic information; request an appointment; open clinic directions
Super Admin	Monitor all doctors and appointments; edit/remove portals; use network-level chatbot
System Administrator	Deploy, secure, monitor and maintain the application and database

CHAPTER 2

Requirements and System Analysis

CARE-GRID: DOCTOR WEBSITE MAKER

2.1 Existing System Analysis

Traditional clinic websites are commonly static and require a developer for every update. Appointment requests may be handled through phone calls, social media messages or generic forms without a unified administrative record. These approaches create fragmented information, inconsistent branding, limited visibility and weak operational tracking.

2.2 Proposed System

Care-Grid proposes a multi-portal platform where the same application can generate and operate multiple doctor websites. Each doctor receives a unique slug-based public route and a protected dashboard. A super administrator receives an aggregated view of doctor portals and patient appointment requests.

2.3 Functional Requirements

ID	Requirement	Priority
FR-01	Doctor submits verified details and creates a website	High
FR-02	System generates a unique public doctor URL	High
FR-03	Patient submits appointment request	High
FR-04	Doctor views and updates appointment status	High
FR-05	Super admin views doctors and appointments	High
FR-06	Doctor updates profile, content, theme and media	Medium
FR-07	Doctor and super admin use data-grounded chatbot	Medium
FR-08	OTP verifies registration, login and password recovery	High
FR-09	Google Maps searches clinic name and address	Medium

Table 2.1: Functional requirements

2.4 Non-Functional Requirements

Category	Requirement
Security	Hash passwords and OTPs; enforce role authorization; apply CORS, Helmet, validation and rate limiting
Usability	Provide responsive interfaces, custom feedback messages and simple guided flows
Reliability	Persist records in MongoDB and preserve existing Cloudinary assets when replacement fails
Performance	Use indexed database fields, bounded payloads and limited chatbot context
Maintainability	Organize code into routes, controllers, services, models, validations and reusable React components
Deployability	Support Vercel frontend, Render backend and MongoDB Atlas with environment-based configuration

Table 2.2: Non-functional requirements

2.5 Feasibility Study

- Technical feasibility: The MERN stack and selected managed services support all required workflows.
- Operational feasibility: Guided forms and dashboards reduce the need for specialized technical skills.
- Economic feasibility: Vercel, Render, MongoDB Atlas and Cloudinary offer entry-level managed hosting options.
- Schedule feasibility: Modular development enables features to be implemented and tested incrementally.

CHAPTER 3

System Design, Architecture and Database

CARE-GRID: DOCTOR WEBSITE MAKER

3.1 Architecture

Care-Grid follows a three-layer web architecture. The React client handles presentation and interactions. The Express API applies validation, business logic, authentication and integrations. MongoDB and external services persist data and provide media, email and AI capabilities.

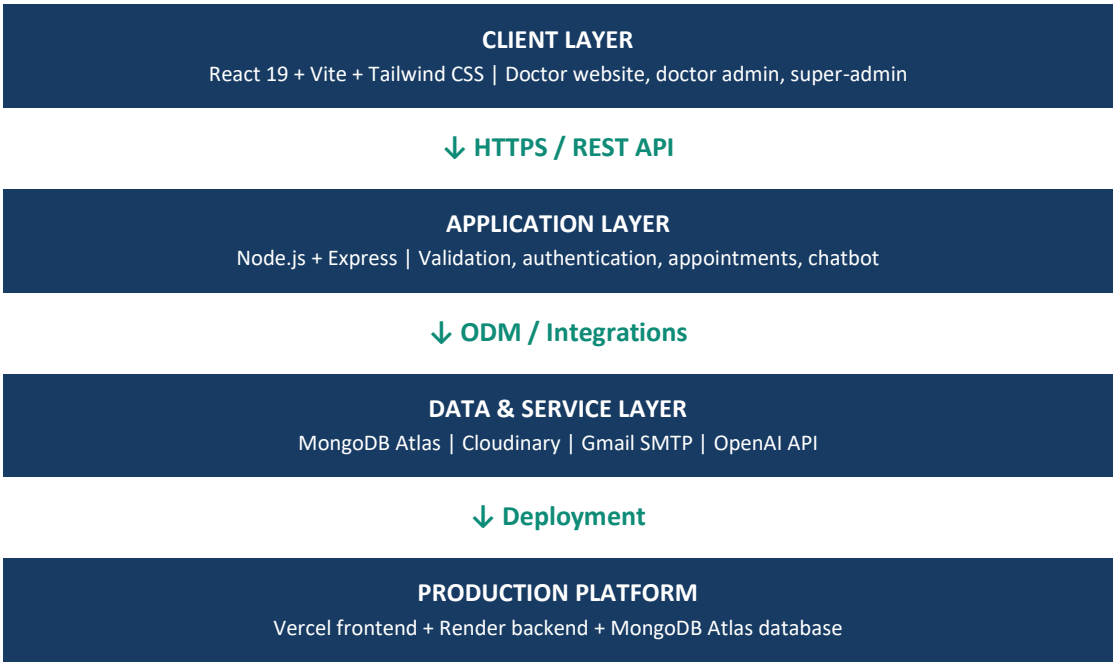

Figure 3.1: High-level Care-Grid system architecture

3.2 Doctor Website Generation Workflow

Step	Doctor onboarding workflow	System action
1	Fill website creation form	Validate structured doctor, clinic and media data
2	Verify registered email using OTP	Issue purpose-bound, hashed, expiring OTP challenge
3	Generate website	Upload media, generate AI content and store doctor record
4	Publish doctor portal	Expose public doctor route using a unique slug
5	Manage operations	Doctor admin handles appointments, content and chatbot

Figure 3.2: Doctor website generation workflow

3.3 Database Design

MongoDB is used because the platform stores nested doctor profiles, flexible AI-generated content, appointment snapshots, OTP metadata and chat message arrays. Mongoose schemas enforce required fields, allowed values, indexes and relationships.

Collection	Important fields	Purpose
Doctor	slug, theme, password, authVersion, personalDetails, media, aiContent	Stores doctor portal and protected account data
Appointment	doctor, doctorSnapshot, patient, treatment, date,	Stores patient appointment requests

Collection	Important fields	Purpose
	timeSlot, status	
Admin	email, password	Stores super-admin credentials
OtpChallenge	accountType, purpose, accountId, otpHash, expiresAt, attempts	Stores purpose-bound one-time verification challenges
ChatConversation	ownerType, owner, messages	Persists doctor and super-admin chatbot history

Table 3.1: MongoDB collections

3.4 Key Relationships and Indexes

- Each appointment references one doctor using the doctor ObjectId.
- Chat conversations use a unique ownerType and owner combination.
- Doctor slugs and doctor email addresses are unique.
- Appointment doctor/date/status fields are indexed for administrative queries.
- OTP expiry uses a TTL index so expired challenges are automatically removed.

3.5 External Service Design

Service	Use in Care-Grid
Cloudinary	Secure profile and clinic gallery media hosting
Gmail SMTP / Nodemailer	OTP and password-change notification email delivery
OpenAI API	Doctor website content and grounded administrative assistance
Google Maps Search	Clinic directions using clinic name, address and country
MongoDB Atlas	Managed production database

CHAPTER 4

Implementation

CARE-GRID: DOCTOR WEBSITE MAKER

4.1 Technology Stack

Layer	Technology	Responsibility
Frontend	React 19, Vite, Tailwind CSS, Axios	Responsive pages, components, forms and API communication
Backend	Node.js, Express 5	REST API, authentication, validation and integrations
Database	MongoDB, Mongoose	Persistent structured application data
Authentication	JWT, bcrypt, email OTP	Role access, password hashing and identity verification
Operations	Vercel, Render, MongoDB Atlas	Production hosting and managed database

4.2 Frontend Modules

- Home: doctor onboarding form, theme selection, image previews, discard controls and registration OTP.
- Doctor Profile: public responsive doctor website, gallery, treatments, contact details, maps and appointment modal.
- Doctor Admin Panel: secure login, OTP, forgot password, analytics, appointments, website editor and chatbot.
- Super Admin Login and Dashboard: secure OTP login, doctor directory, appointments, analytics, editing and chatbot.
- Reusable components: OTP verification, file upload, gallery slider, chatbot, appointment manager, analytics and UI provider.

4.3 Backend Modules

- Routes define public, doctor-protected and super-admin-protected endpoints.
- Controllers implement doctor, appointment, authentication, admin and chatbot workflows.
- Services isolate OTP, SMTP, Cloudinary, OpenAI and grounded chatbot behavior.
- Mongoose models define persistent data structures and indexes.
- Joi validations reject malformed or unsafe request data before controller logic.
- Middleware handles authentication, upload validation, error responses, logging and authorization.

4.4 Core Application Routes

Method	Route	Purpose
POST	/api/auth/registration-otp	Send registration verification code
POST	/api/auth/verify-otp	Verify purpose-bound OTP
POST	/api/doctors/register	Create verified doctor portal
GET	/api/doctors/:slug	Load public doctor website
POST	/api/doctors/:slug/appointments	Create patient appointment
POST	/api/doctors/:slug/login	Verify doctor credentials and send OTP
GET	/api/doctors/:slug/session	Validate doctor session and load profile

Method	Route	Purpose
GET	/api/admin/doctors	Load all doctor portals for super admin
GET	/api/health	Report backend and database health

Table 4.1: Core application routes

4.5 OTP Authentication Flow

1. The user submits valid credentials or a registration email.
2. The backend generates a cryptographically random six-digit OTP.
3. Only the bcrypt hash of the OTP is saved with purpose, expiry and attempt count.
4. The OTP is delivered through Gmail SMTP.
5. The user enters the OTP using the verification interface.
6. The backend verifies purpose, expiry, attempts and hash, then deletes the challenge.
7. A purpose-specific short-lived token or role access token is issued.

4.6 Example Configuration

```
Frontend environment:
VITE_API_URL=https://care-grid-api.onrender.com

Backend deployment:
Root Directory: server
Build Command: npm ci --omit=dev
Start Command: npm start
Health Check Path: /api/health
```

CHAPTER 5

Security, Validation and Error Handling

CARE-GRID: DOCTOR WEBSITE MAKER

5.1 Security Approach

Care-Grid applies defense in depth. No individual control is treated as sufficient. Input validation, authentication, authorization, secure hashing, rate limiting, origin restrictions, payload limits and production secret management work together to reduce risk.

Control	Implementation	Risk reduced
Password storage	bcrypt hashing	Plaintext credential exposure
OTP storage	Purpose-bound bcrypt hashes, expiry and attempt limit	OTP theft, replay and cross-flow misuse
Authorization	JWT roles and protected middleware	Unauthorized dashboard access
Session invalidation	Doctor authVersion increment after reset	Continued use of old sessions
Validation	Joi schemas and Mongoose rules	Malformed and inconsistent records
Rate limiting	Global and sensitive-route limits	Brute force and request abuse
HTTP security	Helmet, CORS allowlist and hidden framework signature	Common browser and origin attacks
Upload security	Size, count, MIME and magic-byte validation	Spoofed or oversized file uploads
Secret management	Environment variables and Git ignore rules	Credential leakage

Table 5.1: Security controls

5.2 Validation Rules

- Doctor mobile number must contain exactly ten digits.
- Doctor registration email must be verified and unique.
- Passwords require uppercase, lowercase, number and special character.
- Appointment status is restricted to pending, confirmed, completed or cancelled.
- Uploaded media is restricted to JPEG, PNG or WebP and maximum file/count limits.
- Chat messages and history have bounded length.

5.3 Error Handling

The API uses a centralized error middleware to produce consistent JSON responses. Expected validation and authentication errors return client-appropriate status codes. Unexpected errors are logged without returning stack traces or secrets. The frontend displays custom feedback rather than browser alert boxes.

Condition	HTTP status	Response behavior
Invalid form or JSON	400	Return concise validation message
Missing or invalid session	401	Reject request and require login
Wrong role or CORS origin	403	Reject unauthorized access
Missing record or route	404	Return not-found response
Duplicate unique record	409	Return conflict response

Condition	HTTP status	Response behavior
Service temporarily unavailable	503	Return safe retry message

5.4 Privacy Considerations

The administrative chatbot receives a minimized data context. Patient phone numbers, patient email addresses and free-form patient messages are not included in the OpenAI context. The chatbot is instructed to answer only from administrative data and not provide medical diagnosis or treatment advice.

CHAPTER 6

Testing and Quality Assurance

CARE-GRID: DOCTOR WEBSITE MAKER

6.1 Testing Strategy

The project was tested through static checks, production builds, dependency audits, schema validation tests, live API regression checks, database audits and production-mode startup verification. Testing focused on both expected workflows and invalid or malicious inputs.

Test area	Representative check	Result
Frontend quality	ESLint across React source	Passed
Frontend production	Vite production build	Passed
Backend syntax	Syntax check for all backend JavaScript files	Passed
Dependency security	npm audit --omit=dev for client and server	0 vulnerabilities
Health check	GET /api/health with connected database	200 healthy
Authorization	Unauthenticated admin request	401 rejected
CORS	Request from unauthorized origin	403 rejected
Validation	Eleven-digit doctor mobile	400 rejected
Input parsing	Malformed JSON request	400 rejected
Upload security	Spoofed JPEG file	400 rejected
Database	Unique doctor email index and record scan	Passed
Production startup	Render-style NODE_ENV=production smoke test	Passed

Table 6.1: Testing summary

6.2 Sample Test Cases

ID	Scenario	Expected result
TC-01	Doctor registration without OTP token	Registration rejected
TC-02	Reuse a successfully verified OTP	Second verification rejected
TC-03	Doctor login with correct password	OTP sent before dashboard access
TC-04	Password reset followed by old JWT use	Old session rejected
TC-05	Patient books appointment	Record visible to doctor and super admin
TC-06	Doctor opens maps direction	Clinic name and address searched together
TC-07	Replace image upload fails	Existing image remains available

6.3 Database Quality Findings

The production-readiness audit confirmed that doctor email addresses have a unique database index, OTP challenge storage was empty after testing, and appointment and doctor collections were accessible. A legacy doctor record containing a twelve-digit phone number was identified; the system now rejects such values, and the legacy record must be corrected using the actual ten-digit number rather than unsafe automatic truncation.

CHAPTER 7

Deployment, Conclusion and Future Scope

CARE-GRID: DOCTOR WEBSITE MAKER

7.1 Production Deployment Design

Care-Grid is prepared for a split deployment model. The React/Vite frontend is deployed on Vercel, the Node/Express backend is deployed on Render, and MongoDB Atlas provides the production database. Cloudinary, Gmail SMTP and OpenAI remain managed external integrations.

Platform	Configuration
GitHub	Source repository with secrets, node_modules, logs and builds excluded
Vercel	Root directory client; VITE_API_URL points to Render service
Render	Root directory server; npm ci --omit=dev; npm start; /api/health
MongoDB Atlas	SRV connection string, dedicated user and controlled network access

7.2 Required Production Environment

```
NODE_ENV=production
MONGO_URI=<MongoDB Atlas SRV URI>
CLIENT_URLS=<Vercel production URL>
JWT_SECRET=<long random secret>
OPENAI_API_KEY=<secret>
CLOUDINARY_CLOUD_NAME / CLOUDINARY_API_KEY / CLOUDINARY_API_SECRET
EMAIL_USER / EMAIL_APP_PASSWORD
ADMIN_EMAIL / ADMIN_PASSWORD (initial bootstrap only)
```

7.3 Conclusion

Care-Grid successfully demonstrates how a modern MERN application can simplify doctor website creation while supporting real operational workflows. The platform connects public doctor websites, patient appointment booking, doctor administration and centralized supervision through a single database-driven system.

The completed implementation goes beyond a basic website generator by integrating secure OTP authentication, password recovery, responsive themes, media management, grounded AI assistance, analytics dashboards, persistent chat history, validation, production deployment configuration and security hardening.

7.4 Future Scope

- Add doctor subscription plans, billing and custom domains.
- Add verified patient accounts and appointment reminders.
- Add calendar integration and precise time-slot availability.
- Add multilingual doctor websites and accessibility enhancements.
- Add audit logs, admin activity history and advanced reporting.
- Add secure teleconsultation while maintaining healthcare privacy requirements.
- Add automated database backups, uptime monitoring and incident alerts.

BIBLIOGRAPHY AND REFERENCES

1. React Documentation. https://react.dev/
2. Vite Documentation. https://vite.dev/
3. Node.js Documentation. https://nodejs.org/docs/latest/api/
4. Express Documentation. https://expressjs.com/
5. MongoDB and Mongoose Documentation. https://www.mongodb.com/docs/ and https://mongoosejs.com/docs/
6. OWASP Cheat Sheet Series: Authentication, Password Storage and Forgot Password. https://cheatsheetseries.owasp.org/
7. Nodemailer SMTP Documentation. https://nodemailer.com/smtp
8. Cloudinary Documentation. https://cloudinary.com/documentation
9. OpenAI API Documentation. https://platform.openai.com/docs/
10. Vercel Vite Deployment Documentation. https://vercel.com/docs/frameworks/vite
11. Render Node/Express Deployment Documentation. https://render.com/docs/deploy-node-express-app
12. MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/

APPENDIX A: PROJECT DIRECTORY STRUCTURE

```
doctor-portal-maker/
  client/
    src/components/       Reusable UI components
    src/pages/            Public, doctor-admin and super-admin pages
    src/utils/            Client authentication utilities
    vercel.json           SPA rewrite configuration
  server/
    controllers/          Business workflow handlers
    middlewares/          Authentication, validation, uploads and errors
    models/               MongoDB/Mongoose schemas
    routes/               REST API routes
    services/             Email, OTP, Cloudinary and AI integrations
    validations/          Joi request schemas
  render.yaml             Render deployment blueprint
  PRODUCTION_DEPLOYMENT.md
```

APPENDIX B: USER GUIDE

For Doctors

1. Open the Care-Grid website creation form and enter valid professional and clinic information.
2. Upload a profile image and optional clinic gallery images.
3. Verify the submitted email using the received OTP.
4. Open the generated public website and doctor admin URL.
5. Sign in using registered credentials and complete OTP verification.
6. Use the dashboard to manage appointments, website content, media, theme and chatbot.

For Patients

1. Open the doctor's public Care-Grid website.
2. Review doctor, clinic, treatments, gallery and directions information.
3. Select Book Appointment and submit the required information.
4. Wait for the clinic to confirm or update the appointment request.

For Super Admin

1. Open /superadmin and sign in using the authorized account.
2. Complete email OTP verification.
3. Review all doctors, appointments and network analytics.
4. Edit or remove doctor portals when authorized.
5. Use the grounded chatbot for administrative questions.